

Key Engine — Design & Architecture

1. Purpose of the Key Engine

The Key Engine is a deterministic normalization layer responsible for detecting and rewriting semantic keys in raw transaction narratives into canonical key forms.

Note: It works only on narratives with delimiters viz swift,ach,wire,generic

Its **sole responsibility** is to:

- **Identify key-like phrases** (labels) and normalize them into a canonical vocabulary that parsers can rely on.

It does not:

- Extract values
- Assign payer or payee
- Perform business inference
- Guess intent

It strictly answers the question: "Is this text segment a key, and if so, what canonical key does it represent?"

2. Why the Key Engine Exists

Banking narratives are inconsistent across:

- Banks
- Products
- Regions
- Message types

The same semantic field can appear as:

BNF, BENEF, BENEFICIARY, ULTI BENE, RECEIVER NAME

Embedding this variation directly into parsers would:

- Inflate parser complexity
- Cause duplication
- Increase false positives
- Make maintenance unsafe

The Key Engine centralizes this problem.

Parsers downstream can operate on a stable, canonical key vocabulary, regardless of how the original narrative expressed it.

3. Canonical Key Model

Canonical keys are defined externally in: `canonical_keys.json` file.

The structure is conceptually as follows:

Canonical Key	Acceptable Surface-Form Variants
beneficiary	bnf, beneficiary, ultimate beneficiary, receiver
originator	orig, originator, sending co name

This file defines:

- The canonical semantic key (e.g., beneficiary)
- All acceptable surface-form variants

The Key Engine does not hardcode key logic — updates are data-driven.

4. High-Level Algorithm

At a high level, the Key Engine performs five steps:

1. Normalize input text
2. Generate candidate key windows
3. Match windows to canonical variants
4. Resolve overlapping conflicts
5. Rewrite the narrative with canonical keys

Each step is deterministic and explainable.

5. Input Normalization

Before any detection:

- Text is lowercased
- Whitespace is normalized
- No punctuation stripping is done globally

None

```
text = text.lower()
text = re.sub(r"\\s+", " ", text)
```

This ensures stable tokenization while preserving positional offsets.

6. Sliding Window Generation

The engine tokenizes the text and generates sliding windows of up to `max_words` (default: 7).

Example input: `sending co name = applied systems`

Generated windows include:

- sending
- sending co
- sending co name
- co name
- name

Each window tracks:

- Phrase text
- Start offset
- End offset
- Token count

This enables multi-word key detection without regex explosion.

7. Delimiter Awareness (Critical Design Choice)

A phrase is only considered a key candidate if it is followed by a delimiter:

This avoids false positives such as:

- Company names
- Addresses
- Free text

For example:

- `sending co name = value` ✓ **key**
- `sending co name value` ✗ **ignored**

It is important to note that the Key Engine is only being used for delimiter-based narratives, which currently include:

- Wire
- ACH
- SWIFT
- Generic delimiter-based formats

This design choice dramatically reduces noise.

8. Canonical Matching Logic

Each candidate window is matched against all known key variants using fuzzy similarity:

- RapidFuzz ratio
- Threshold-based (default: 85%)

Comparison is done on cleaned strings:

- Lowercased
- Alphanumeric only

Only the best-scoring canonical variant is retained per window. This allows detection of:

- `benficiary` → `beneficiary`
- `sendng co nm` → `sending co name`

without permitting arbitrary drift.

9. Conflict Resolution Strategy for overlapping matches

Overlapping matches are resolved deterministically using two rules:

Rule 1 — Higher confidence wins

The match with the higher fuzzy score is retained.

Rule 2 — Longer phrase wins on tie

If scores tie, the match covering more tokens is preferred.

This ensures:

- `sending co name` beats `co name`
- `ultimate beneficiary` beats `beneficiary`

No randomness. No order dependence.

10. Text Rewriting

Accepted matches are **applied backwards (right to left)** to preserve offsets. Each detected key phrase is replaced with its canonical form.

Example:

`SENDING CO NAME = APPLIED SYSTEMS`

Rewritten as:

`originator = APPLIED SYSTEMS`

This rewritten text is what downstream parsers consume.

11. Human-in-the-Loop (HITL) Support

The Key Engine explicitly supports controlled human review, without guessing. Two HITL mechanisms exist:

1. Ambiguous near-matches

If a phrase is similar to known keys but not accepted, it can be surfaced for review.

2. Completely unknown keys

If a previously unseen key: pattern is detected:

- It is collected
- Suggested variants are generated

- Returned for manual inspection

No automatic learning occurs. Nothing is silently added.

12. Output Contract

None

The public entry point:

```
rewritten_text, hitl_flag = rewrite(text)
```

Returns:

- Rewritten canonical text
- HITL metadata if unknown keys were detected

This allows:

- Parsers to proceed safely
- UI / tooling to flag unknown key patterns

13. Design Principles (Why This Works)

The Key Engine is built around a few strict principles:

- Determinism over inference
- Rewrite, don't extract
- Delimiter-aware detection
- Centralized vocabulary
- Human-review over auto-learning

This keeps it:

- Predictable
- Auditable
- Safe for financial data

14. What the Key Engine Does Not Do (By Design)

The engine's current scope of usage **does not** include the following responsibilities:

- Inferring semantic roles
- Assigning payer/payee
- Parsing values
- Modifying numbers or identifiers
- Automatic learning

15. Relationship to Other Components

The Key Engine is a **supporting normalization layer, not a decision engine.**

- Parsers depend on the Key Engine for stable key names
- CTPTY depends on parser output, not key detection
- Alias Engine operates after payer/payee extraction
- UI/API never call the Key Engine directly

16. Summary

The Key Engine is a focused, deterministic subsystem that transforms inconsistent, bank-specific key labels into a canonical vocabulary. It enables parser simplicity, reduces duplication, and provides a controlled pathway for vocabulary expansion through human review.